PropOS Phase 1 数据模型文档

> 版本: v1.3
>
> 日期: 2026-04-08
>
> 对应 **PRD**: v1.7
>
> 范围: Phase 1 五个核心模块

一、总览

1.1 实体关系层级

```
users (→ departments)                                  ← 公共：认证与角色
departments                                            ← 公共：三级组织树（公司→部门→组）
user_managed_scopes (→ departments, users, buildings, floors) ← 公共：管辖范围
audit_logs                                             ← 公共：审计日志
job_execution_logs                                     ← 公共：任务执行与补偿

buildings → floors → units                             ← M1 资产
                        └── renovation_records

tenants → contracts → contract_attachments             ← M2 租务
                    └── contract_units (← units)       ← M2 合同-单元 M:N
                    └── rent_escalation_phases
                    └── deposits → deposit_transactions  ← M2 押金
                    └── alerts
                    └── invoices → invoice_items
                    └── payments → payment_allocations
                    └── subleases (← units)            ← M5 二房东

meter_readings (→ units)                               ← M3 水电抄表
turnover_reports (→ contracts)                         ← M3 商铺营业额对账
import_batches                                         ← 公共：导入批次追踪

expenses (→ buildings, units?)                         ← M3 财务支出
noi_budgets (→ buildings?)                               ← M3 NOI 预算
kpi_metric_definitions                                 ← M3 KPI 指标库
kpi_schemes → kpi_scheme_metrics                      ← M3 KPI 考核方案
            → kpi_scheme_targets (→ departments, users) ← M3 KPI 方案绑定对象
kpi_score_snapshots → kpi_score_snapshot_items      ← M3 KPI 快照
                    → kpi_appeals                     ← M3 KPI 申诉

suppliers                                              ← M4 工单
work_orders (→ units, buildings, floors, users)     ← M4 工单
   └── work_order_photos
```

1.2 跨模块 FK 关系汇总

表	外键字段	引用表	说明
floors	building_id	buildings	楼层归属楼栋
units	floor_id , building_id	floors , buildings	单元归属楼层/楼栋
renovation_records	unit_id	units	改造记录归属单元
contracts	tenant_id	tenants	合同绑定租客
contracts	parent_contract_id	contracts	续签合同链
contract_units	contract_id , unit_id	contracts , units	合同-单元 M:N 关联（含计费面积与单价）
contract_attachments	contract_id	contracts	合同附件
rent_escalation_phases	contract_id	contracts	租金递增阶段
deposits	contract_id	contracts	押金归属合同
deposit_transactions	deposit_id	deposits	押金流水审计
alerts	contract_id	contracts	预警记录归属合同
invoices	contract_id	contracts	账单归属合同
invoice_items	invoice_id	invoices	账单明细
payments	received_by_user_id	users	收款主记录
payment_allocations	payment_id , invoice_id	payments , invoices	收款核销分配
expenses	building_id , unit_id ?	buildings , units	运营支出归口
kpi_scheme_metrics	scheme_id , metric_id	kpi_schemes , kpi_metric_definitions	方案-指标关联
kpi_score_snapshots	scheme_id , evaluated_user_id	kpi_schemes , users	打分快照
kpi_score_snapshot_items	snapshot_id , metric_id	kpi_score_snapshots , kpi_metric_definitions	快照明细
work_orders	unit_id , floor_id , building_id	units , floors , buildings	工单定位
work_orders	reporter_user_id , assignee_user_id , supplier_id	users , users , suppliers	工单人员
work_order_photos	work_order_id	work_orders	工单照片
subleases	master_contract_id , unit_id	contracts , units	子租赁关联主合同与单元

subleases	reviewer_user_id , submitted_by_user_id	users , users	填报/审核人
meter_readings	unit_id , recorded_by	units , users	水电抄表归属单元
turnover_reports	contract_id , reviewed_by	contracts , users	商铺营业额申报
import_batches	created_by	users	导入批次操作人
noi_budgets	building_id	buildings	NOI 预算关联楼标（可选）
audit_logs	user_id	users	操作人
departments	parent_id	departments	组织树父级
users	department_id	departments	员工归属部门
user_managed_scopes	department_id , user_id	departments , users	管辖范围归属
user_managed_scopes	building_id , floor_id	buildings , floors	管辖资产引用
kpi_scheme_targets	scheme_id , user_id , department_id	kpi_schemes , users , departments	方案绑定对象
kpi_appeals	snapshot_id , appellant_id , reviewer_id	kpi_score_snapshots , users , users	KPI 申诉

二、PostgreSQL 自定义枚举类型

```
-- 业态
CREATE TYPE property_type AS ENUM ('office', 'retail', 'apartment');

-- 单元出租状态
CREATE TYPE unit_status AS ENUM ('leased', 'vacant', 'expiring_soon', 'non_leasable');

-- 单元装修状态
CREATE TYPE unit_decoration AS ENUM ('blank', 'simple', 'refined', 'raw');

-- 用户角色
CREATE TYPE user_role AS ENUM (
    'super_admin',         -- 超级管理员
    'operations_manager', -- 运营管理层
    'leasing_specialist', -- 租务专员
    'finance_staff',       -- 财务人员
    'frontline_staff',     -- 前线员工
    'sub_landlord'         -- 二房东
);
```

```
-- 租客类型
CREATE TYPE tenant_type AS ENUM ('corporate', 'individual');

-- 合同状态机
CREATE TYPE contract_status AS ENUM (
    'quoting',         -- 报价中
    'pending_sign',    -- 待签约
    'active',          -- 执行中
    'expiring_soon',   -- 即将到期（≤90天）
    'expired',         -- 已到期
    'renewed',         -- 已续签
    'terminated'       -- 已终止
);

-- 租金递增类型
CREATE TYPE escalation_type AS ENUM (
    'fixed_rate',            -- 固定比例递增
    'fixed_amount',          -- 固定金额递增
    'step',                  -- 阶梯式递增
    'cpi',                   -- CPI 挂钩递增
    'periodic',              -- 每 N 年递增
    'base_after_free_period' -- 免租后基准调整
);

-- 预警类型
CREATE TYPE alert_type AS ENUM (
    'lease_expiry_90',          -- 到期预警 90 天
    'lease_expiry_60',          -- 到期预警 60 天
    'lease_expiry_30',          -- 到期预警 30 天
    'payment_overdue_1',        -- 逾期第 1 天
    'payment_overdue_7',        -- 逾期第 7 天
    'payment_overdue_15',       -- 逾期第 15 天
    'monthly_expiry_summary',   -- 月度到期汇总
    'deposit_refund_reminder'   -- 押金退还提醒（终止前 7 天）
);

-- 账单状态
CREATE TYPE invoice_status AS ENUM (
    'draft',     -- 草稿（生成中）
    'issued',    -- 已出账
    'paid',      -- 已核销
    'overdue',   -- 逾期
    'cancelled', -- 已作废
    'exempt'     -- 免租期免单
);

-- 账单费项类型
```

```sql
CREATE TYPE invoice_item_type AS ENUM (
    'rent',           -- 租金
    'management_fee', -- 物管费
    'electricity',    -- 电费
    'water',          -- 水费
    'parking',        -- 停车费
    'storage',        -- 储藏室
    'revenue_share',  -- 营业额分成（商铺）
    'other'           -- 其他
);

-- 运营支出类目
CREATE TYPE expense_category AS ENUM (
    'utility_common',      -- 水电公摊
    'outsourced_property', -- 外包物业费
    'repair',              -- 维修费
    'insurance',           -- 保险
    'tax',                 -- 税金
    'other'                -- 其他
);

-- 工单类型
CREATE TYPE work_order_type AS ENUM (
    'repair',             -- 报修
    'complaint',          -- 投诉
    'inspection'          -- 退租验房
    -- Phase 2 预留: 'patrol', 'cleaning', 'maintenance', 'security', 'renovation_request'
);

-- 工单状态
CREATE TYPE work_order_status AS ENUM (
    'submitted',          -- 已提交
    'approved',           -- 已审核/派单
    'in_progress',        -- 处理中
    'pending_inspection', -- 待验收
    'completed',          -- 已完成
    'rejected',           -- 已拒绝
    'on_hold'             -- 挂起
);

-- 工单紧急程度
CREATE TYPE work_order_priority AS ENUM ('normal', 'urgent', 'critical');

-- 子租赁入住状态
CREATE TYPE sublease_occupancy_status AS ENUM (
    'occupied',           -- 已入住
    'signed_not_moved',   -- 已签约未入住
    'moved_out',          -- 已退租
```

```
    'vacant'              -- 空置
);

-- 子租赁审核状态
CREATE TYPE sublease_review_status AS ENUM (
    'draft',     -- 草稿（二房东填报未提交）
    'pending',   -- 待审核
    'approved',  -- 已通过
    'rejected'   -- 已退回
);

-- KPI 评估周期
CREATE TYPE kpi_period_type AS ENUM ('monthly', 'quarterly', 'yearly');

-- 押金状态（v1.7 新增）
CREATE TYPE deposit_status AS ENUM (
    'collected',           -- 已收取
    'frozen',              -- 冻结中
    'partially_credited',  -- 部分冲抵
    'refunded'             -- 已退还
);

-- 合同终止类型（v1.7 新增）
CREATE TYPE termination_type AS ENUM (
    'normal_expiry',       -- 正常到期
    'tenant_early_exit',   -- 租户提前退租
    'mutual_agreement',    -- 协商提前终止
    'owner_termination'    -- 业主单方解约
);

-- 水电表类型（v1.7 新增）
CREATE TYPE meter_type AS ENUM ('water', 'electricity', 'gas');

-- 抄表周期（v1.7 新增）
CREATE TYPE reading_cycle AS ENUM ('monthly', 'bimonthly');

-- 营业额申报审核状态（v1.7 新增）
CREATE TYPE turnover_approval_status AS ENUM ('pending', 'approved', 'rejected');

-- 导入数据类别（v1.7 新增）
CREATE TYPE import_data_type AS ENUM ('units', 'contracts', 'invoices');

-- 导入回滚状态（v1.7 新增）
CREATE TYPE import_rollback_status AS ENUM ('committed', 'rolled_back');

-- 信用评级（v1.7 新增）
CREATE TYPE credit_rating AS ENUM ('A', 'B', 'C');
```

> **v1.7 建模约束**: KPI 表保留完整结构，但业务语义为"试运行评分"；财务核销改为"收款主记录 + 分配明细"双表，以支持部分收款和跨账单核销；合同-单元改为 M:N 关联（通过 `contract_units` ），每个单元独立记录计费面积与单价；新增押金独立建账、水电抄表、营业额对账、导入批次追踪。

三、公共基础表

3.1 users（用户）

```
CREATE TABLE users (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(100) NOT NULL,
    email             VARCHAR(255) UNIQUE NOT NULL,
    password_hash     TEXT        NOT NULL,            -- bcrypt hash
    role              user_role   NOT NULL,
    -- 二房东角色专用：绑定主合同（允许 NULL，非二房东角色为 NULL）
    -- FK: ALTER TABLE users ADD CONSTRAINT fk_users_contract
    --       FOREIGN KEY (bound_contract_id) REFERENCES contracts(id);
    bound_contract_id UUID,
    is_active         BOOLEAN     NOT NULL DEFAULT TRUE,
    failed_login_attempts SMALLINT NOT NULL DEFAULT 0,
    locked_until      TIMESTAMPTZ,
    password_changed_at TIMESTAMPTZ,
    last_login_at     TIMESTAMPTZ,
    session_version   INTEGER     NOT NULL DEFAULT 1,
    -- 主合同到期后二房东账号自动冻结
    frozen_at         TIMESTAMPTZ,
    frozen_reason     TEXT,
    -- 员工归属部门（KPI 正式考核依赖）
    department_id     UUID,                            -- FK → departments(id)，延迟建约
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_role       ON users(role);
CREATE INDEX idx_users_email      ON users(email);
CREATE INDEX idx_users_contract   ON users(bound_contract_id) WHERE bound_contract_id IS NOT
NULL;
CREATE INDEX idx_users_locked_until ON users(locked_until) WHERE locked_until IS NOT NULL;
CREATE INDEX idx_users_department ON users(department_id) WHERE department_id IS NOT NULL;
```

3.2 audit_logs（操作审计日志）

覆盖范围（架构约束 #4）：合同变更、账单核销、权限变更、二房东数据提交。

```
CREATE TABLE audit_logs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID        NOT NULL REFERENCES users(id),
    action        VARCHAR(100) NOT NULL, -- 'contract.update', 'invoice.write_off',
'user.role_change', 'sublease.submit'
    resource_type VARCHAR(50)  NOT NULL, -- 'contract', 'invoice', 'user', 'sublease'
```

```
    resource_id   UUID        NOT NULL,
    -- 变更前后快照（JSON）
    before_json   JSONB,
    after_json    JSONB,
    ip_address    INET,
    retention_until TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_user        ON audit_logs(user_id);
CREATE INDEX idx_audit_resource    ON audit_logs(resource_type, resource_id);
CREATE INDEX idx_audit_created_at  ON audit_logs(created_at DESC);
```

3.3 refresh_tokens（刷新令牌）

JWT access token 过期后通过 refresh token 续签，支持单设备吊销和会话版本校验。

```
CREATE TABLE refresh_tokens (
    id             UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id        UUID          NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash     TEXT          NOT NULL UNIQUE,  -- SHA-256 hash 存储，不保留明文
    device_info    VARCHAR(200),                   -- 设备标识（User-Agent 摘要）
    session_version INTEGER      NOT NULL,          -- 签发时的 users.session_version，改密/冻结后
旧 token 失效
    expires_at     TIMESTAMPTZ   NOT NULL,
    revoked_at     TIMESTAMPTZ,                    -- 非 NULL 表示已吊销
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_refresh_tokens_user    ON refresh_tokens(user_id);
CREATE INDEX idx_refresh_tokens_expires ON refresh_tokens(expires_at) WHERE revoked_at IS NULL;
```

3.4 job_execution_logs（任务执行日志）

用于记录账单生成、预警推送、催收提醒、导入后处理等定时任务的执行结果，支持失败重试与人工补偿。

```
CREATE TABLE job_execution_logs (
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_name         VARCHAR(100) NOT NULL,
    job_scope        VARCHAR(100),
    status           VARCHAR(20)  NOT NULL, -- 'running', 'success', 'failed',
'retry_scheduled'
    retry_count      SMALLINT     NOT NULL DEFAULT 0,
    started_at       TIMESTAMPTZ  NOT NULL,
    finished_at      TIMESTAMPTZ,
    error_message    TEXT,
    payload_json     JSONB,
    triggered_by_user_id UUID REFERENCES users(id),
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_job_logs_name_status ON job_execution_logs(job_name, status);
```

```
CREATE INDEX idx_job_logs_started_at  ON job_execution_logs(started_at DESC);
```

四、M1 资产与空间可视化

4.1 buildings（楼栋）

```
CREATE TABLE buildings (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(100) NOT NULL,           -- 'A座', '商铺区', '公寓楼'
    property_type property_type NOT NULL,          -- 主业态（楼栋整体定性）
    total_floors  SMALLINT     NOT NULL,
    gfa           NUMERIC(10,2) NOT NULL,          -- 总建筑面积（m²）
    nla           NUMERIC(10,2) NOT NULL,          -- 净可租面积（m²）
    address       TEXT,
    built_year    SMALLINT,
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
```

4.2 floors（楼层）

```
CREATE TABLE floors (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id   UUID         NOT NULL REFERENCES buildings(id) ON DELETE CASCADE,
    floor_number  SMALLINT     NOT NULL,           -- 负数代表地下层（B1=-1）
    floor_name    VARCHAR(50),                     -- 展示名，如 'B1', '1F', '10F'
    -- CAD 转换后的文件路径（floors/{building_id}/{floor_id}.svg）
    svg_path      TEXT,
    png_path      TEXT,
    nla           NUMERIC(10,2),                   -- 本层净可租面积（m²）
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    UNIQUE (building_id, floor_number)
);

CREATE INDEX idx_floors_building ON floors(building_id);
```

4.2.1 floor_plans（楼层图纸版本管理）

支持同一楼层保留多个版本图纸（改造前/后），`floors.svg_path` / `png_path` 始终指向当前生效版本。

```
CREATE TABLE floor_plans (
    id            UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    floor_id      UUID          NOT NULL REFERENCES floors(id) ON DELETE CASCADE,
    version_label VARCHAR(50)   NOT NULL,           -- 如 '原始图纸', '2026年改造后'
    svg_path      TEXT          NOT NULL,           -- floors/{building_id}/{floor_id}_v{n}.svg
    png_path      TEXT,
    is_current    BOOLEAN       NOT NULL DEFAULT FALSE,
```

```
    uploaded_by   UUID          REFERENCES users(id),
    created_at    TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

-- 同一楼层仅一个版本为当前生效
CREATE UNIQUE INDEX uq_floor_plan_current
    ON floor_plans(floor_id)
    WHERE is_current = TRUE;

CREATE INDEX idx_floor_plans_floor ON floor_plans(floor_id);
```

4.3 units（单元/房源）

单元是 PropOS 的核心资产原子，承载三业态差异化字段，通过 `ext_fields` JSONB 存储业态专属属性。

```
CREATE TABLE units (
    id                UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    floor_id          UUID         NOT NULL REFERENCES floors(id),
    building_id       UUID         NOT NULL REFERENCES buildings(id),
    unit_number       VARCHAR(50)  NOT NULL,        -- 如 '10A', '501'
    property_type     property_type NOT NULL,
    -- 面积
    gross_area        NUMERIC(10,2),                -- 建筑面积（m²）
    net_area          NUMERIC(10,2),                -- 套内面积（m²）
    -- 基础属性
    orientation       VARCHAR(20),                  -- 朝向：'east', 'south', 'west', 'north'
    ceiling_height    NUMERIC(4,2),                 -- 层高（m）
    decoration_status unit_decoration NOT NULL DEFAULT 'blank',
    -- 出租状态（由 unit_service 根据合同状态实时计算，冗余存储用于快速查询）
    current_status    unit_status  NOT NULL DEFAULT 'vacant',
    -- 是否可租（非可租如公共区域、设备间）
    is_leasable       BOOLEAN      NOT NULL DEFAULT TRUE,
    -- 业态扩展字段（JSONB，结构见下方说明）
    ext_fields        JSONB        NOT NULL DEFAULT '{}',
    -- 当前绑定合同（冗余存储，提升楼层色块查询性能）
    current_contract_id UUID,
    -- QR 码标识（用于扫码报修）
    qr_code           VARCHAR(100) UNIQUE,
    -- 参考市场租金（元/m²/月），由运营定期维护，用于空置损失测算与 PGI 估算（v1.7 新增）
    market_rent_reference NUMERIC(10,2),
    -- 前序单元 ID 列表，记录单元拆分/合并/停租/转非可租的历史（v1.7 新增）
    predecessor_unit_ids  UUID[],
    -- 归档时间，旧单元标记为 archived 而非物理删除（v1.7 新增）
    archived_at       TIMESTAMPTZ,
    created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    UNIQUE (building_id, unit_number)
);

CREATE INDEX idx_units_floor        ON units(floor_id);
CREATE INDEX idx_units_building     ON units(building_id);
```

```
CREATE INDEX idx_units_status       ON units(current_status);
CREATE INDEX idx_units_type         ON units(property_type);
CREATE INDEX idx_units_ext_fields   ON units USING GIN(ext_fields);
```

`ext_fields` JSONB 结构说明

业态	字段结构
`office` （写字楼）	`{"workstation_count": 20, "partition_count": 3}`
`retail` （商铺）	`{"frontage_width": 8.5, "street_facing": true, "retail_ceiling_height": 5.2}`
`apartment` （公寓）	`{"bedroom_count": 2, "en_suite_bathroom": true, "occupant_count": null}`

4.4 renovation_records（改造记录）

```
CREATE TABLE renovation_records (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    unit_id           UUID         NOT NULL REFERENCES units(id) ON DELETE CASCADE,
    renovation_type   VARCHAR(100) NOT NULL, -- '隔断改造', '水电改造', '装修升级' 等
    started_at        DATE         NOT NULL,
    completed_at      DATE,
    cost              NUMERIC(12,2),          -- 施工造价（元）
    contractor        VARCHAR(200),           -- 施工方
    description       TEXT,
    -- 照片路径（renovations/{record_id}/{index}.jpg）
    -- 存储在 work_order_photos 之外的独立文件，路径约定见文件存储规范
    before_photo_paths TEXT[],               -- 改造前照片路径数组
    after_photo_paths  TEXT[],              -- 改造后照片路径数组
    created_by        UUID         REFERENCES users(id),
    created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_renovation_unit ON renovation_records(unit_id);
```

五、M2 租务与合同管理

5.1 tenants（租客）

安全约束：`id_number_encrypted` 与 `contact_phone_encrypted` 使用 AES-256 加密存储，API 层默认返回脱敏值（后4位）。

```
CREATE TABLE tenants (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_type   tenant_type NOT NULL,
```

```
    -- 企业名称或个人姓名
    display_name  VARCHAR(200) NOT NULL,
    -- [加密存储] 企业：统一社会信用代码；个人：身份证号
    -- 存储值为 AES-256-GCM 密文（base64）
    id_number_encrypted    TEXT,       -- 加密：AES-256-GCM
    -- [加密存储] 主要联系电话
    contact_phone_encrypted TEXT,      -- 加密：AES-256-GCM
    -- 企业联系人姓名（企业租客必填）
    contact_person         VARCHAR(100),
    contact_email          VARCHAR(255),
    emergency_contact_name  VARCHAR(100),
    emergency_contact_phone TEXT,       -- 加密：AES-256-GCM
    -- 信用评级（系统自动计算：A/B/C）
    credit_rating   CHAR(1) CHECK (credit_rating IN ('A','B','C')),
    overdue_count   SMALLINT NOT NULL DEFAULT 0, -- 历史逾期次数（用于信用计算）
    -- 信用评级计算辅助字段（v1.7 新增）
    -- 评级规则：A（优质）= 12 个月内逾期 ≤1 次且单次 ≤3 天；
    --          B（一般）= 12 个月内逾期 2~3 次或单次 4~15 天；
    --          C（风险）= 12 个月内逾期 ≥4 次或单次 >15 天
    -- 每月 1 日自动重算；新租户默认 B 级；签约满 3 个月后首次评级
    last_rating_date          DATE,             -- 最近一次评级日期
    times_overdue_past_12m    SMALLINT NOT NULL DEFAULT 0, -- 12 个月内逾期次数
    max_single_overdue_days   SMALLINT NOT NULL DEFAULT 0, -- 单次最长逾期天数
    notes           TEXT,
    -- PIPL 合规：合同终止后个人信息保留不超过 3 年，过期后脱敏处理
    data_retention_until TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_tenants_type ON tenants(tenant_type);
CREATE INDEX idx_tenants_retention ON tenants(data_retention_until) WHERE data_retention_until
IS NOT NULL;
```

5.2 contracts（合同）

```
CREATE TABLE contracts (
    id                UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 合同编号（业务可读编号，格式如 'C-2026-001'）
    contract_no       VARCHAR(50)    UNIQUE NOT NULL,
    tenant_id         UUID           NOT NULL REFERENCES tenants(id),
    status            contract_status NOT NULL DEFAULT 'quoting',
    property_type     property_type  NOT NULL, -- 冗余，与单元业态一致，便于聚合查询

    -- 合同期限
    start_date        DATE           NOT NULL,
    end_date          DATE           NOT NULL,
    -- 免租/装修期
    free_rent_days    SMALLINT       NOT NULL DEFAULT 0,
```

```
    free_rent_end_date DATE,                    -- 免租结束日（start_date + free_rent_days - 1）

    -- 租金
    base_monthly_rent  NUMERIC(12,2)  NOT NULL, -- 签约基准月租金（元）
    -- 付款周期（月数，如 1=月付，3=季付，6=半年付，12=年付）
    payment_cycle_months SMALLINT     NOT NULL DEFAULT 1,
    -- 物管费（元/m²/月，公寓业态可为 0 表示含于租金）
    management_fee_rate  NUMERIC(8,4) NOT NULL DEFAULT 0,
    -- 押金（月数，押金总额 = 押金月数 × 月租金）
    deposit_months       SMALLINT     NOT NULL DEFAULT 2,
    deposit_amount       NUMERIC(12,2) NOT NULL,

    -- 税费口径（v1.7 新增）
    tax_inclusive     BOOLEAN        NOT NULL DEFAULT TRUE,  -- 含税/不含税标识
    applicable_tax_rate NUMERIC(5,4) NOT NULL DEFAULT 0,    -- 适用税率（如 0.09 = 9%，0.05 = 5%）

    -- 商铺营业额分成（revenue_share_enabled=true 时有效）
    revenue_share_enabled  BOOLEAN    NOT NULL DEFAULT FALSE,
    min_guarantee_rent     NUMERIC(12,2),       -- 保底租金（元/月）
    revenue_share_rate     NUMERIC(5,4),        -- 分成比例（如 0.08 = 8%）

    -- 续签链
    parent_contract_id     UUID       REFERENCES contracts(id),
    is_sublease_master     BOOLEAN    NOT NULL DEFAULT FALSE, -- true 表示该合同是二房东主合同

    -- 合同 PDF 附件路径（contracts/{id}/signed.pdf）
    -- 详见 contract_attachments 表
    signed_pdf_path  TEXT,

    -- 合同终止信息（v1.7 增强：四种终止类型）
    termination_type       termination_type,     -- 正常到期/租户提前退租/协商提前终止/业主单方解约
    terminated_at          TIMESTAMPTZ,
    termination_date       DATE,                 -- 实际终止日期
    termination_reason     TEXT,                 -- 解约依据/补偿方案
    penalty_amount         NUMERIC(12,2),        -- 违约金（元）
    deposit_deduction_details TEXT,              -- 押金扣除明细

    created_by       UUID           REFERENCES users(id),
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_contract_dates CHECK (start_date <= end_date)
);

CREATE INDEX idx_contracts_tenant   ON contracts(tenant_id);
CREATE INDEX idx_contracts_status   ON contracts(status);
CREATE INDEX idx_contracts_end_date ON contracts(end_date) WHERE status IN
('active','expiring_soon');
CREATE INDEX idx_contracts_master   ON contracts(is_sublease_master) WHERE is_sublease_master =
TRUE;
```

5.3 contract_attachments（合同附件）

```
CREATE TABLE contract_attachments (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id   UUID        NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
    -- 文件类型：'original'=原合同，'amendment'=补充协议，'other'=其他
    file_type     VARCHAR(50) NOT NULL DEFAULT 'original',
    filename      VARCHAR(255) NOT NULL,
    -- 存储路径：contracts/{contract_id}/{filename}
    storage_path  TEXT        NOT NULL,
    file_size_kb  INTEGER,
    uploaded_by   UUID        REFERENCES users(id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_attachments_contract ON contract_attachments(contract_id);
```

5.4 contract_units（合同-单元 M:N 关联）

v1.7 变更：合同与单元由 1:1 改为 M:N 关系。每个关联记录独立记录计费面积与单价，WALE 计算中多单元合同按单元拆分后分别计入，避免重复加权。

```
CREATE TABLE contract_units (
    contract_id   UUID         NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
    unit_id       UUID         NOT NULL REFERENCES units(id),
    -- 该单元计费面积（m²），可能与建筑面积不同
    billing_area  NUMERIC(10,2) NOT NULL,
    -- 该单元租金单价（元/m²/月）
    unit_price    NUMERIC(10,2) NOT NULL,
    PRIMARY KEY (contract_id, unit_id)
);

CREATE INDEX idx_contract_units_unit ON contract_units(unit_id);
```

5.5 deposits（押金）

v1.7 新增：押金独立建账，不计入 NOI 收入。每次状态变更需记录原因和审批人，所有变更写入 `deposit_transactions` 审计表。续签时押金可整体转移至新合同（无需先退后收）。

状态机：`collected → frozen → partially_credited → refunded`

```
CREATE TABLE deposits (
    id                UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id       UUID           NOT NULL REFERENCES contracts(id),
    amount            NUMERIC(12,2)  NOT NULL, -- 押金金额（元）
    collection_date   DATE           NOT NULL, -- 收取日期
    status            deposit_status NOT NULL DEFAULT 'collected',
    last_status_change_at TIMESTAMPTZ,
    -- 转移至新合同（续签时整体转移）
    transferred_to_contract_id UUID  REFERENCES contracts(id),
```

```
    notes             TEXT,
    created_by        UUID           REFERENCES users(id),
    created_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_deposits_contract ON deposits(contract_id);
CREATE INDEX idx_deposits_status   ON deposits(status);
```

5.6 deposit_transactions（押金流水审计）

```
CREATE TABLE deposit_transactions (
    id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    deposit_id      UUID           NOT NULL REFERENCES deposits(id) ON DELETE CASCADE,
    -- 流水类型：收取/冻结/扣除/退还/转移
    transaction_type VARCHAR(20)   NOT NULL CHECK (transaction_type IN
                       ('collection','freeze','deduction','refund','transfer')),
    amount          NUMERIC(12,2)  NOT NULL, -- 本次操作金额
    previous_status deposit_status NOT NULL, -- 变更前状态
    new_status      deposit_status NOT NULL, -- 变更后状态
    reason          TEXT           NOT NULL, -- 状态变更原因
    approved_by     UUID           REFERENCES users(id), -- 审批人
    created_by      UUID           NOT NULL REFERENCES users(id), -- 操作人
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_deposit_tx_deposit ON deposit_transactions(deposit_id);
```

5.7 rent_escalation_phases（租金递增阶段）

每份合同可配置多个递增阶段，顺序执行。对应 `rent_escalation_engine` package 的输入结构。

```
CREATE TABLE rent_escalation_phases (
    id             UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id    UUID           NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
    phase_order    SMALLINT       NOT NULL, -- 阶段顺序，从 1 开始
    -- 阶段起止（相对合同起租日的月数偏移，0 表示起租日当月）
    start_month    SMALLINT       NOT NULL DEFAULT 0,
    end_month      SMALLINT,              -- NULL 表示延续至合同结束
    escalation_type escalation_type NOT NULL,
    -- 类型专属参数（JSONB，结构见下方说明）
    params         JSONB          NOT NULL,
    created_at     TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    UNIQUE (contract_id, phase_order)
);

CREATE INDEX idx_escalation_contract ON rent_escalation_phases(contract_id);
```

`params` JSONB 结构说明

```
// fixed_rate: 固定比例递增
{ "rate": 0.05, "interval_months": 12 }

// fixed_amount: 固定金额递增（元/m²/月）
{ "amount_per_sqm": 3.0, "interval_months": 12 }

// step: 阶梯式递增
{ "steps": [
    { "from_month": 0,  "to_month": 23, "monthly_rent": 8000 },
    { "from_month": 24, "to_month": 47, "monthly_rent": 9000 },
    { "from_month": 48, "to_month": null, "monthly_rent": 10000 }
]}

// cpi: CPI 挂钩递增
{ "interval_months": 12, "cpi_year_overrides": { "2027": 0.023, "2028": 0.031 } }

// periodic: 每 N 年递增
{ "interval_years": 2, "rate": 0.08 }

// base_after_free_period: 免租后基准调整
{ "base_monthly_rent": 7500 }
```

5.7.1 escalation_templates（递增规则模板）

PRD 要求支持"保存为模板"和"从模板快速应用"，减少重复配置。模板独立于合同存在，合同签约时可从模板复制递增阶段。

```
CREATE TABLE escalation_templates (
    id              UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    template_name   VARCHAR(100)    NOT NULL,
    property_type   property_type   NOT NULL,        -- 三业态标识，便于筛选
    description     TEXT,
    -- 模板包含的递增阶段（JSONB 数组，结构同 rent_escalation_phases.params）
    phases          JSONB           NOT NULL,         -- [{"phase_order":1,
"escalation_type":"fixed_rate", "params":{...}}, ...]
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    created_by      UUID            REFERENCES users(id),
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_escalation_tpl_type ON escalation_templates(property_type);
```

5.8 alerts（预警记录）

```
CREATE TABLE alerts (
    id           UUID       PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id  UUID       NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
    alert_type   alert_type NOT NULL,
    -- 预警触发日期（调度任务写入）
    triggered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    -- 目标用户（NULL 表示按角色广播，非 NULL 表示定向推送）
    target_user_id UUID     REFERENCES users(id),
    -- 是否已读/处理
    is_read      BOOLEAN    NOT NULL DEFAULT FALSE,
    read_by      UUID       REFERENCES users(id),
    read_at      TIMESTAMPTZ,
    -- 通知状态
    notified_via TEXT[],    -- ['in_app', 'email']
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_alerts_contract   ON alerts(contract_id);
CREATE INDEX idx_alerts_target     ON alerts(target_user_id) WHERE target_user_id IS NOT NULL;
CREATE INDEX idx_alerts_unread     ON alerts(is_read, triggered_at DESC) WHERE is_read = FALSE;
```

六、M3 财务与 NOI

6.1 invoices（账单）

```
CREATE TABLE invoices (
    id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 账单编号（如 'INV-2026-04-001'）
    invoice_no      VARCHAR(50)    UNIQUE NOT NULL,
    contract_id     UUID           NOT NULL REFERENCES contracts(id),
    -- 账期起止
    period_start    DATE           NOT NULL,
    period_end      DATE           NOT NULL,
    -- 应收总额（各费项之和）
    total_amount    NUMERIC(12,2)  NOT NULL,
    paid_amount     NUMERIC(12,2)  NOT NULL DEFAULT 0,
    outstanding_amount NUMERIC(12,2) NOT NULL,
    status          invoice_status NOT NULL DEFAULT 'issued',
    billing_basis   VARCHAR(30)    NOT NULL DEFAULT 'contract', -- 'contract',
'daily_prorated', 'fixed_total'
    tax_mode        VARCHAR(20)    NOT NULL DEFAULT 'net',      -- 'net', 'gross'
    -- 发票信息
    invoice_issued  BOOLEAN        NOT NULL DEFAULT FALSE,
    invoice_no_ext  VARCHAR(100),             -- 外部发票号
    invoice_issued_at TIMESTAMPTZ,
    -- 逾期信息
    due_date        DATE           NOT NULL,  -- 缴款截止日
    overdue_since   DATE,                     -- 首次标记逾期日
    -- 催收记录
    last_reminded_at TIMESTAMPTZ,
    -- 商铺营业额（仅 revenue_share_enabled 合同使用）
    reported_revenue NUMERIC(12,2),           -- 本期申报营业额
    created_by      UUID           REFERENCES users(id),
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
```

```
    updated_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_invoice_period CHECK (period_start <= period_end),
    CONSTRAINT chk_invoice_amounts CHECK (
        total_amount >= 0 AND paid_amount >= 0 AND outstanding_amount >= 0
    )
);

CREATE INDEX idx_invoices_contract   ON invoices(contract_id);
CREATE INDEX idx_invoices_status     ON invoices(status);
CREATE INDEX idx_invoices_due_date   ON invoices(due_date) WHERE status IN
('issued','overdue');
CREATE INDEX idx_invoices_period     ON invoices(period_start, period_end);
```

6.2 invoice_items（账单费项明细）

```
CREATE TABLE invoice_items (
    id          UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
    invoice_id  UUID              NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
    item_type   invoice_item_type NOT NULL,
    description VARCHAR(200),              -- 费项说明（如 '10月电费'）
    quantity    NUMERIC(10,4),             -- 数量（面积/度数/月数）
    unit        VARCHAR(20),               -- 单位（'m²', 'kWh', '月'）
    unit_price  NUMERIC(10,4),             -- 单价
    amount      NUMERIC(12,2)  NOT NULL,   -- 金额（元）
    created_at  TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_invoice_items_invoice ON invoice_items(invoice_id);
```

6.3 payments（收款核销）

```
CREATE TABLE payments (
    id             UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    paid_amount    NUMERIC(12,2) NOT NULL,
    paid_at        TIMESTAMPTZ   NOT NULL, -- 实际到账时间（UTC）
    -- 核销方式：'bank_transfer', 'cash', 'online', 'offset'（冲抵）
    payment_method VARCHAR(50)  NOT NULL DEFAULT 'bank_transfer',
    reference_no   VARCHAR(100),            -- 银行流水号/交易单号
    -- 核销人
    received_by_user_id UUID   REFERENCES users(id),
    notes          TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_payments_paid_at ON payments(paid_at DESC);
CREATE INDEX idx_payments_reference ON payments(reference_no) WHERE reference_no IS NOT NULL;
```

6.4 payment_allocations（收款核销分配）

```
CREATE TABLE payment_allocations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id      UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,
    invoice_id      UUID NOT NULL REFERENCES invoices(id),
    allocated_amount NUMERIC(12,2) NOT NULL,
    allocated_by_user_id UUID REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_payment_allocation_amount CHECK (allocated_amount > 0),
    UNIQUE (payment_id, invoice_id)
);

CREATE INDEX idx_payment_allocations_payment ON payment_allocations(payment_id);
CREATE INDEX idx_payment_allocations_invoice ON payment_allocations(invoice_id);
```

6.5 expenses（运营支出）

支出关联楼栋（必填）和单元（可选，用于工单维修成本归口），自动汇入 NOI 运营支出端。

```
CREATE TABLE expenses (
    id            UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id   UUID              NOT NULL REFERENCES buildings(id),
    -- 可选关联至单元（工单维修费用精确归口）
    unit_id       UUID              REFERENCES units(id),
    -- 可选关联至工单（维修成本由工单触发）
    work_order_id UUID,             -- FK 延迟建立（引用 work_orders）
    category      expense_category NOT NULL,
    description   TEXT              NOT NULL,
    amount        NUMERIC(12,2)     NOT NULL,  -- 金额（元）
    expense_date  DATE              NOT NULL,
    vendor        VARCHAR(200),                -- 供应商/服务商名称
    -- 发票/收据凭证
    receipt_path  TEXT,                        -- 存储路径
    created_by    UUID              REFERENCES users(id),
    created_at    TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_expenses_building  ON expenses(building_id);
CREATE INDEX idx_expenses_date      ON expenses(expense_date);
CREATE INDEX idx_expenses_category  ON expenses(category);
CREATE INDEX idx_expenses_workorder ON expenses(work_order_id) WHERE work_order_id IS NOT NULL;
```

6.6 meter_readings（水电抄表记录）

v1.7 新增：支持水/电/气三种表计。业态差异：写字楼/商铺独立分表；公寓视合同约定（含于租金或独立）。录入抄表数据后自动生成水电费账单（附带用量明细）。支持阶梯水电价（按用量分段配价）和公共区域公摊（按面积比例分摊）。

```
CREATE TABLE meter_readings (
    id                UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    unit_id           UUID        NOT NULL REFERENCES units(id),
    meter_type        meter_type  NOT NULL,             -- water / electricity / gas
    reading_cycle     reading_cycle NOT NULL DEFAULT 'monthly',
    -- 读数
    current_reading   NUMERIC(12,2) NOT NULL,           -- 本期读数
    previous_reading  NUMERIC(12,2) NOT NULL,           -- 上期读数
    consumption       NUMERIC(12,2) NOT NULL,           -- 用量 = current - previous
    -- 计费
    unit_price        NUMERIC(10,4) NOT NULL,           -- 元/度 或 元/吨
    cost_amount       NUMERIC(12,2) NOT NULL,           -- 费用 = consumption × unit_price
    -- 阶梯计价明细（可选，用于阶梯水电价）
    tiered_details    JSONB,                            --
[{"from":0,"to":100,"price":0.5,"amount":50}, ...]
    -- 抄表信息
    reading_date      DATE        NOT NULL,
    recorded_by       UUID        REFERENCES users(id),
    -- 是否已生成对应账单
    invoice_generated BOOLEAN     NOT NULL DEFAULT FALSE,
    generated_invoice_id UUID     REFERENCES invoices(id),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_meter_unit       ON meter_readings(unit_id);
CREATE INDEX idx_meter_date       ON meter_readings(reading_date DESC);
CREATE INDEX idx_meter_type       ON meter_readings(meter_type);
CREATE INDEX idx_meter_uninvoiced ON meter_readings(invoice_generated) WHERE invoice_generated
= FALSE;
```

6.7 turnover_reports（商铺营业额申报）

v1.7 新增：商铺营业额分成对账流程。商户按月提交营业额 → 财务审核 → 系统自动生成分成账单。支持补报/修正（差额账单自动生成）和争议处理记录。

```
CREATE TABLE turnover_reports (
    id                UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id       UUID        NOT NULL REFERENCES contracts(id),
    -- 申报月份
    report_month      DATE        NOT NULL,             -- 月份（yyyy-mm-01）
    -- 营业额数据
    reported_revenue NUMERIC(12,2) NOT NULL,            -- 申报营业额
    revenue_share_rate NUMERIC(5,4) NOT NULL,           -- 分成比例
    base_rent         NUMERIC(12,2) NOT NULL,           -- 保底租金
    calculated_share NUMERIC(12,2) NOT NULL,            -- 计算分成额 = MAX(reported_revenue ×
rate - base_rent, 0)
    -- 审核
    approval_status   turnover_approval_status NOT NULL DEFAULT 'pending',
    reviewed_by       UUID        REFERENCES users(id),
    reviewed_at       TIMESTAMPTZ,
```

```
    rejection_reason TEXT,
    -- 附件（POS 流水或审计报表）
    attachment_paths TEXT[],
    -- 是否为补报/修正
    is_amendment     BOOLEAN     NOT NULL DEFAULT FALSE,
    original_report_id UUID     REFERENCES turnover_reports(id),
    -- 生成的账单
    generated_invoice_id UUID   REFERENCES invoices(id),
    -- 争议记录
    dispute_note     TEXT,
    submitted_by     UUID        REFERENCES users(id),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 同一合同同一月份仅一条正式记录（补报/修正记录不受此约束）
CREATE UNIQUE INDEX uq_turnover_original
    ON turnover_reports(contract_id, report_month)
    WHERE is_amendment = FALSE;

CREATE INDEX idx_turnover_contract ON turnover_reports(contract_id);
CREATE INDEX idx_turnover_month    ON turnover_reports(report_month);
CREATE INDEX idx_turnover_status   ON turnover_reports(approval_status);
```

6.7.1 noi_budgets（NOI 预算）

v1.7 新增：录入各楼栋或业态的 NOI 预算值，用于 K07（NOI 达成率）指标计算。匹配规则：取最近一条 `building_id`（或 `property_type`）+ `period_year` + `period_month`（可选）匹配的记录为预算基准。

```
CREATE TABLE noi_budgets (
    id              UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 可按楼栋或业态维度录入，两者均为可选（同时为 NULL 表示全局预算）
    building_id     UUID            REFERENCES buildings(id),
    property_type   property_type,
    -- 预算周期
    period_year     SMALLINT        NOT NULL,  -- 预算年份，如 2026
    period_month    SMALLINT        CHECK (period_month BETWEEN 1 AND 12),  -- NULL 表示年度预算
    -- 预算值
    budget_noi      NUMERIC(14,2)   NOT NULL,  -- 预算 NOI（元）
    -- 录入人
    created_by      UUID            REFERENCES users(id),
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_noi_budgets_building ON noi_budgets(building_id) WHERE building_id IS NOT
NULL;
CREATE INDEX idx_noi_budgets_type     ON noi_budgets(property_type) WHERE property_type IS NOT
NULL;
CREATE INDEX idx_noi_budgets_period   ON noi_budgets(period_year, period_month);
```

> K07 取数逻辑：`actual_noi / budget_noi`，其中 `budget_noi` 取匹配到当前考核周期的最近一条预算记录。实际 NOI 展现实现（received）口径：EGI - OpEx。

6.8 kpi_metric_definitions（KPI 指标定义库）

系统预定义 10 个指标（K01-K10），由初始化脚本 seed，不允许用户新增。Phase 1 中这些指标用于正式 KPI 考核评分。

```
CREATE TABLE kpi_metric_definitions (
    id                  UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    code                VARCHAR(10) UNIQUE NOT NULL, -- 'K01' ~ 'K10'
    name                VARCHAR(100) NOT NULL,
    description         TEXT,
    -- 默认满分/及格/红线阈值（各方案可覆盖）
    default_full_score_threshold  NUMERIC(10,4) NOT NULL, -- 如 0.95（95%）
    default_pass_threshold        NUMERIC(10,4) NOT NULL, -- 及格线
    default_fail_threshold        NUMERIC(10,4) NOT NULL, -- 不及格红线（0分起点）
    -- 指标值越大越好（TRUE）还是越小越好（FALSE，如逾期率、空置周转天数）
    higher_is_better    BOOLEAN     NOT NULL DEFAULT TRUE,
    -- 指标方向标识（v1.7 新增，与 higher_is_better 语义一致，便于前端展示）
    -- 'positive'=数值越高越好（K01/K02/K04/K07/K09/K10）
    -- 'negative'=数值越低越好（K03/K05/K06/K08），线性插值逻辑翻转
    direction           VARCHAR(10) NOT NULL DEFAULT 'positive'
                        CHECK (direction IN ('positive', 'negative')),
    -- 数据来源模块
    source_module       VARCHAR(50) NOT NULL, -- 'assets', 'contracts', 'finance', 'workorders'
    -- 是否允许手动录入（K10 租户满意度）
    is_manual_input     BOOLEAN     NOT NULL DEFAULT FALSE,
    is_enabled          BOOLEAN     NOT NULL DEFAULT TRUE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

初始化数据（Seed）

code	name	full_score	pass	fail	higher_is_better	direction
K01	出租率	0.95	0.80	0.60	TRUE	positive
K02	收款及时率	0.95	0.85	0.70	TRUE	positive
K03	租户集中度	0.40	0.55	0.70	FALSE	negative
K04	续约率	0.80	0.60	0.40	TRUE	positive
K05	工单响应时效（小时，仅 repair）	24	48	72	FALSE	negative
K06	空置周转天数	30	60	90	FALSE	negative
K07	NOI 达成率	1.00	0.85	0.70	TRUE	positive

K08	逾期率	0.05	0.15	0.20	FALSE	negative
K09	租金递增执行率	0.95	0.85	0.70	TRUE	positive
K10	租户满意度（手动）	90	75	60	TRUE	positive

6.9 kpi_schemes（KPI 方案）

```
CREATE TABLE kpi_schemes (
    id           UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    name         VARCHAR(200)    NOT NULL, -- '租务部考核方案 2026'
    period_type  kpi_period_type NOT NULL,
    -- 方案有效期（支持版本迭代，旧方案数据保留）
    effective_from DATE          NOT NULL,
    effective_to   DATE,                     -- NULL 表示持续有效
    is_active    BOOLEAN         NOT NULL DEFAULT TRUE,
    scoring_mode VARCHAR(20)     NOT NULL DEFAULT 'official', -- 'trial', 'official'
    created_by   UUID            REFERENCES users(id),
    created_at   TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);
```

6.10 kpi_scheme_metrics（方案-指标关联）

业务约束：同一方案下所有指标 `weight` 之和必须 = 1.00，在 Service 层校验。

```
CREATE TABLE kpi_scheme_metrics (
    id          UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    scheme_id   UUID         NOT NULL REFERENCES kpi_schemes(id) ON DELETE CASCADE,
    metric_id   UUID         NOT NULL REFERENCES kpi_metric_definitions(id),
    weight      NUMERIC(5,4) NOT NULL CHECK (weight > 0 AND weight <= 1),
    -- 本方案中覆盖默认阈值（NULL 表示使用 kpi_metric_definitions 的默认值）
    full_score_threshold NUMERIC(10,4),
    pass_threshold       NUMERIC(10,4),
    fail_threshold       NUMERIC(10,4),
    UNIQUE (scheme_id, metric_id)
);

CREATE INDEX idx_scheme_metrics_scheme ON kpi_scheme_metrics(scheme_id);
```

6.11 kpi_score_snapshots（KPI 打分快照）

```
CREATE TABLE kpi_score_snapshots (
    id                UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    scheme_id         UUID        NOT NULL REFERENCES kpi_schemes(id),
    evaluated_user_id UUID        NOT NULL REFERENCES users(id),
    -- 评估时间范围
    period_start      DATE        NOT NULL,
    period_end        DATE        NOT NULL,
    -- 汇总总分（0-100）
```

```
    total_score       NUMERIC(5,2) NOT NULL,
    snapshot_status   VARCHAR(20) NOT NULL DEFAULT 'draft',  -- 'draft', 'frozen',
'recalculated'
    frozen_at         TIMESTAMPTZ,                           -- 冻结时间（触发申诉窗口 7 日计时起点）
    calculated_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    created_by        UUID        REFERENCES users(id)
);

CREATE INDEX idx_kpi_snapshots_user   ON kpi_score_snapshots(evaluated_user_id);
CREATE INDEX idx_kpi_snapshots_scheme ON kpi_score_snapshots(scheme_id);
CREATE INDEX idx_kpi_snapshots_period ON kpi_score_snapshots(period_start, period_end);
```

6.12 kpi_score_snapshot_items（打分快照明细）

```
CREATE TABLE kpi_score_snapshot_items (
    id           UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    snapshot_id  UUID         NOT NULL REFERENCES kpi_score_snapshots(id) ON DELETE CASCADE,
    metric_id    UUID         NOT NULL REFERENCES kpi_metric_definitions(id),
    weight       NUMERIC(5,4) NOT NULL,           -- 快照时权重（防止方案修改影响历史）
    actual_value NUMERIC(12,4),                   -- 指标实际值
    score        NUMERIC(5,2) NOT NULL,            -- 本指标得分（0-100）
    weighted_score NUMERIC(5,2) NOT NULL,          -- 加权得分（score × weight × 100）
    source_note  TEXT                              -- 取数说明（便于下钻核查）
);

CREATE INDEX idx_snapshot_items_snapshot ON kpi_score_snapshot_items(snapshot_id);
```

6.13 kpi_scheme_targets（KPI 方案绑定对象）

将 KPI 方案绑定到具体部门或员工，一个方案可绑定多个对象。

```
CREATE TABLE kpi_scheme_targets (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    scheme_id     UUID NOT NULL REFERENCES kpi_schemes(id) ON DELETE CASCADE,
    user_id       UUID REFERENCES users(id),
    department_id UUID REFERENCES departments(id),
    CHECK (user_id IS NOT NULL OR department_id IS NOT NULL)
);

-- 同一方案内每个具体绑定目标唯一（用户级 / 部门级分别约束）
CREATE UNIQUE INDEX uq_scheme_target_user
    ON kpi_scheme_targets(scheme_id, user_id)
    WHERE user_id IS NOT NULL;
CREATE UNIQUE INDEX uq_scheme_target_dept
    ON kpi_scheme_targets(scheme_id, department_id)
    WHERE department_id IS NOT NULL;

CREATE INDEX idx_scheme_targets_scheme ON kpi_scheme_targets(scheme_id);
CREATE INDEX idx_scheme_targets_dept   ON kpi_scheme_targets(department_id) WHERE department_id
IS NOT NULL;
```

6.14 kpi_appeals（KPI 申诉）

员工可在快照冻结后 7 个自然日内提交申诉，管理层审核后决定是否重算。

```
CREATE TABLE kpi_appeals (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    snapshot_id     UUID         NOT NULL REFERENCES kpi_score_snapshots(id),
    appellant_id    UUID         NOT NULL REFERENCES users(id),
    reason          TEXT         NOT NULL,
    status          VARCHAR(20)  NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'approved', 'rejected')),
    reviewer_id     UUID         REFERENCES users(id),
    review_comment  TEXT,
    reviewed_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_kpi_appeals_snapshot ON kpi_appeals(snapshot_id);
CREATE INDEX idx_kpi_appeals_status   ON kpi_appeals(status) WHERE status = 'pending';
```

6.15 departments（组织架构）

三级组织树：公司 → 部门 → 组，通过 `parent_id` 自引用实现层级嵌套。

```
CREATE TABLE departments (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(100) NOT NULL,
    parent_id   UUID REFERENCES departments(id),  -- NULL = 顶级（公司级）
    level       SMALLINT NOT NULL CHECK (level BETWEEN 1 AND 3),
    sort_order  INTEGER  NOT NULL DEFAULT 0,
    is_active   BOOLEAN  NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_departments_parent ON departments(parent_id);
```

6.16 user_managed_scopes（管辖范围）

管辖范围支持部门默认 + 个人覆盖双机制。KPI 数据归集时取个人范围（优先）或继承部门范围。

```
CREATE TABLE user_managed_scopes (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 可绑定到部门（默认范围）或个人（覆盖范围）
    department_id UUID REFERENCES departments(id),
    user_id       UUID REFERENCES users(id),
    -- 管辖维度（至少指定一项）
    building_id   UUID REFERENCES buildings(id),
    floor_id      UUID REFERENCES floors(id),
    property_type property_type,
    CHECK (department_id IS NOT NULL OR user_id IS NOT NULL)
);
```

```
CREATE INDEX idx_managed_scopes_dept ON user_managed_scopes(department_id) WHERE department_id
IS NOT NULL;
CREATE INDEX idx_managed_scopes_user ON user_managed_scopes(user_id) WHERE user_id IS NOT NULL;
```

七、M4 工单系统

7.1 suppliers（供应商）

```
CREATE TABLE suppliers (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(200) NOT NULL,
    -- 供应商类别：'plumbing'=水电，'hvac'=空调，'cleaning'=保洁，'locksmith'=锁具，'other'
    category      VARCHAR(50),
    contact_name  VARCHAR(100),
    contact_phone TEXT,        -- 加密：AES-256-GCM
    address       TEXT,
    notes         TEXT,
    is_active     BOOLEAN     NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.2 work_orders（工单）

```
CREATE TABLE work_orders (
    id                UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 工单编号（如 'WO-2026-04-001'）
    order_no          VARCHAR(50)       UNIQUE NOT NULL,
    -- 工单类型：repair=报修，complaint=投诉，inspection=退租验房
    work_order_type   work_order_type   NOT NULL DEFAULT 'repair',
    -- 位置信息
    building_id       UUID              NOT NULL REFERENCES buildings(id),
    floor_id          UUID              REFERENCES floors(id),
    unit_id           UUID              REFERENCES units(id),
    -- 关联合同（退租验房必填，其他类型可选）
    contract_id       UUID              REFERENCES contracts(id),
    -- 问题描述
    issue_type        VARCHAR(100)      NOT NULL, -- repair: '水电','空调','门窗','消防','网络','电
梯','其他'; complaint: '服务态度','环境噪音','公区卫生','安全隐患','其他'; inspection: '合同到期验
房','提前退租验房'
    priority          work_order_priority NOT NULL DEFAULT 'normal',
    description       TEXT              NOT NULL,
    status            work_order_status  NOT NULL DEFAULT 'submitted',
    -- 人员
    reporter_user_id  UUID              NOT NULL REFERENCES users(id),
    assignee_user_id  UUID              REFERENCES users(id),
    supplier_id       UUID              REFERENCES suppliers(id),
    -- 时间节点
```

```
    submitted_at      TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    approved_at       TIMESTAMPTZ,
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ,
    expected_complete_at TIMESTAMPTZ,
    on_hold_reason    TEXT,
    reopened_from_work_order_id UUID REFERENCES work_orders(id),
    -- 成本（完工后录入，仅 repair 类型适用）
    material_cost     NUMERIC(10,2),    -- 材料费（元）
    labor_cost        NUMERIC(10,2),    -- 人工费（元）
    -- 成本关联通过 expenses.work_order_id 反向引用，无需在此冗余 expense_id
    -- 验收 / 处理结论
    inspection_note   TEXT,             -- repair: 验收备注; complaint: 处理结论; inspection: 查验
结论
    rejected_reason   TEXT,
    -- 退租验房专用字段
    deposit_deduction_suggestion NUMERIC(10,2), -- 建议押金扣减金额（inspection 类型）
    follow_up_work_order_id UUID,       -- 查验后生成的维修跟进工单（inspection → repair）
    -- 来源（报修渠道）
    source            VARCHAR(20) NOT NULL DEFAULT 'app', -- 'app', 'mini_program', 'manual'
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_workorders_building  ON work_orders(building_id);
CREATE INDEX idx_workorders_unit      ON work_orders(unit_id) WHERE unit_id IS NOT NULL;
CREATE INDEX idx_workorders_status    ON work_orders(status);
CREATE INDEX idx_workorders_type      ON work_orders(work_order_type);
CREATE INDEX idx_workorders_reporter  ON work_orders(reporter_user_id);
CREATE INDEX idx_workorders_submitted ON work_orders(submitted_at DESC);
CREATE INDEX idx_workorders_contract  ON work_orders(contract_id) WHERE contract_id IS NOT
NULL;
```

7.3 work_order_photos（工单照片）

```
CREATE TABLE work_order_photos (
    id             UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    work_order_id  UUID        NOT NULL REFERENCES work_orders(id) ON DELETE CASCADE,
    -- 照片时机: 'before'=报修时, 'after'=完工后
    photo_stage    VARCHAR(20) NOT NULL DEFAULT 'before',
    -- 存储路径: workorders/{work_order_id}/{index}.jpg
    storage_path   TEXT        NOT NULL,
    sort_order     SMALLINT    NOT NULL DEFAULT 0,
    uploaded_by    UUID        REFERENCES users(id),
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_workorder_photos_order ON work_order_photos(work_order_id);
```

八、M5 二房东穿透管理

8.1 subleases（子租赁记录）

行级隔离约束（架构约束 #2）： `sublease_repository` 的所有查询必须在 WHERE 子句中附加 `master_contract_id IN (SELECT id FROM contracts WHERE tenant_id = $sub_landlord_tenant_id)` 过滤条件，Repository 层不得省略。

安全约束： `sub_tenant_id_number_encrypted` 与 `sub_tenant_phone_encrypted` 使用 AES-256 加密存储；API 响应默认返回脱敏值（后4位），查看完整信息需二次鉴权。

```
CREATE TABLE subleases (
    id                UUID                  PRIMARY KEY DEFAULT gen_random_uuid(),
    -- 关联二房东的主合同（is_sublease_master = TRUE）
    master_contract_id UUID                NOT NULL REFERENCES contracts(id),
    -- 具体单元（必须在主合同覆盖的面积范围内，业务层校验）
    unit_id           UUID                 NOT NULL REFERENCES units(id),
    -- 终端租客信息
    sub_tenant_name   VARCHAR(200)         NOT NULL,
    sub_tenant_type   tenant_type          NOT NULL DEFAULT 'corporate',
    sub_tenant_contact_person VARCHAR(100),
    -- [加密存储] 终端租客证件号
    sub_tenant_id_number_encrypted TEXT,             -- 加密：AES-256-GCM
    -- [加密存储] 终端租客联系电话
    sub_tenant_phone_encrypted     TEXT,             -- 加密：AES-256-GCM

    -- 子租赁期限
    start_date        DATE                 NOT NULL,
    end_date          DATE                 NOT NULL,
    -- 实际月租金（终端租客支付给二房东的）
    monthly_rent      NUMERIC(12,2)        NOT NULL,
    -- 租金单价（元/m²/月），由应用层根据单元面积反算，不使用 GENERATED COLUMN
    -- 公式：rent_per_sqm = monthly_rent / unit.billing_area
    rent_per_sqm      NUMERIC(8,4),

    -- 入住状态
    occupancy_status  sublease_occupancy_status NOT NULL DEFAULT 'occupied',
    -- 公寓：实际入住人数
    occupant_count    SMALLINT,

    -- 审核流
    review_status     sublease_review_status NOT NULL DEFAULT 'pending',
    reviewer_user_id  UUID                 REFERENCES users(id),
    reviewed_at       TIMESTAMPTZ,
    rejection_reason  TEXT,
    version_no        INTEGER              NOT NULL DEFAULT 1,
    declared_for_month DATE,

    -- 填报信息（双通道）
    -- 'internal'=内部录入, 'sub_landlord'=二房东自助填报, 'excel_import'=批量导入
```

```
    submission_channel VARCHAR(20)          NOT NULL DEFAULT 'internal',
    submitted_by_user_id UUID               REFERENCES users(id),
    submitted_at       TIMESTAMPTZ,
    truth_declared_at  TIMESTAMPTZ,

    notes              TEXT,
    -- PIPL 合规：子租赁终止后终端租客个人信息保留不超过 3 年
    data_retention_until TIMESTAMPTZ,
    created_at         TIMESTAMPTZ          NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ          NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_sublease_dates CHECK (start_date <= end_date)
);

-- 同一单元同一时间只能有一条已审核的在租记录
CREATE UNIQUE INDEX uq_sublease_active_unit
    ON subleases(unit_id)
    WHERE occupancy_status IN ('occupied', 'signed_not_moved')
      AND review_status = 'approved';

CREATE INDEX idx_subleases_master_contract ON subleases(master_contract_id);
CREATE INDEX idx_subleases_unit            ON subleases(unit_id);
CREATE INDEX idx_subleases_review_status   ON subleases(review_status);
CREATE INDEX idx_subleases_occupancy       ON subleases(occupancy_status);
```

九、公共：导入批次追踪

9.1 import_batches（导入批次）

v1.7 新增：用于记录 Excel 批量导入的每个批次。单元台账（主数据）采用**整批回滚**策略（一条出错全部不导入）；历史合同和未结账单采用**部分导入**策略（成功入库，失败返回错误明细）。支持试导入模式（仅校验不入库）、按批次回滚、批量修正。

```
CREATE TABLE import_batches (
    id              UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
    batch_name      VARCHAR(200)      NOT NULL,        -- 导入批次名称/编号
    data_type       import_data_type  NOT NULL,        -- units / contracts / invoices
    total_records   INTEGER           NOT NULL,
    success_count   INTEGER           NOT NULL DEFAULT 0,
    failure_count   INTEGER           NOT NULL DEFAULT 0,
    rollback_status import_rollback_status NOT NULL DEFAULT 'committed',
    -- 错误明细：[{"row":5,"field":"gross_area","error":"面积必须为正数"}, ...]
    error_details   JSONB,
    -- 是否为试导入（仅校验不入库）
    is_dry_run      BOOLEAN           NOT NULL DEFAULT FALSE,
    -- 导入文件路径
    source_file_path TEXT,
    created_by      UUID              REFERENCES users(id),
    created_at      TIMESTAMPTZ       NOT NULL DEFAULT NOW()
```

```
);

CREATE INDEX idx_import_batches_type   ON import_batches(data_type);
CREATE INDEX idx_import_batches_status ON import_batches(rollback_status);
```

十、延迟建立的外键约束

以下 FK 存在循环依赖或建表顺序约束，需在所有表创建完成后执行：

```
-- users.bound_contract_id → contracts（二房东账号绑定主合同）
ALTER TABLE users
    ADD CONSTRAINT fk_users_bound_contract
    FOREIGN KEY (bound_contract_id) REFERENCES contracts(id)
    DEFERRABLE INITIALLY DEFERRED;

-- expenses.work_order_id → work_orders（维修成本关联工单）
ALTER TABLE expenses
    ADD CONSTRAINT fk_expenses_work_order
    FOREIGN KEY (work_order_id) REFERENCES work_orders(id)
    DEFERRABLE INITIALLY DEFERRED;
```

十一、索引策略汇总

场景	关键索引
楼层色块渲染	`units(floor_id, current_status)`
WALE 计算	`contracts(status, end_date)` covering index
逾期账单催收	`invoices(status, due_date) WHERE status IN ('issued','overdue')`
二房东数据隔离	`subleases(master_contract_id)`
工单状态监控	`work_orders(status, submitted_at DESC)`
KPI 快照历史	`kpi_score_snapshots(evaluated_user_id, period_start)`
审计日志查询	`audit_logs(resource_type, resource_id)`
单元扩展字段	`units.ext_fields` GIN 索引（按业态过滤）
楼层图纸版本	`floor_plans(floor_id) WHERE is_current = TRUE`
PIPL 数据保留	`tenants(data_retention_until)` WHERE NOT NULL

十二、数据初始化顺序

```
1. 创建所有 ENUM 类型（含 v1.7 新增：deposit_status, termination_type, meter_type 等）
2. departments（组织架构，KPI 正式考核依赖）
3. buildings → floors → floor_plans → units
4. users（第一个超级管理员，含 department_id）
5. refresh_tokens
6. user_managed_scopes（部门默认管辖范围）
7. kpi_metric_definitions（seed K01~K10，含 direction 字段）
8. escalation_templates（递增规则模板）
9. 延迟 FK 约束（ALTER TABLE）
8. 批量导入：639 套单元（Excel 导入工具，经 import_batches 追踪）
9. 批量导入：楼层 CAD 转换（.dwg → svg_path/png_path）
```

十三、v1.7 建模补充说明

1. 收款核销采用 payments + payment_allocations 双表建模，解决部分收款和跨账单核销场景。
2. users 新增登录失败锁定、会话版本和改密时间字段，用于外部门户安全控制。
3. subleases 增加 version*no*、*declared*for*month*、*truth*declared_at，满足月报制填报和版本留痕。
4. kpi*schemes* 与 *kpi*score*snapshots* 增加考核模式与冻结状态字段，`*scoring*mode` 默认为 `'official'` （正式考核），保留 `'trial'` 向后兼容。
5. job*execution*logs 用于承接任务重试、失败巡检和人工补偿，不将任务可靠性散落在业务表中。
6. **合同-单元改为 M:N**：移除 contracts.unit*id* 单外键，通过 *contract*units 中间表实现多单元关联，每条记录独立记录 billing*area* 与 *unit*price。
7. **组织架构与管辖范围**：departments 三级组织树 + user*managed*scopes 管辖范围，支持部门默认 + 个人覆盖，为 KPI 正式考核提供数据归集依据。
8. **KPI 申诉机制**：kpi_appeals 表支持员工申诉 → 管理层审核 → 重算全流程，全程写审计日志。
9. **KPI 方案绑定**：kpi*scheme*targets 将方案与部门/员工关联，取代原 ARCH.md 中的 VARCHAR department 粗略方案。
10. **押金独立建账**：新增 deposits + deposit_transactions 表，押金不计入 NOI 收入，状态机流转全程审计。
11. **水电抄表**：新增 meter_readings 表，支持水/电/气三种表计，阶梯计价与公摊分摊。
12. **商铺营业额对账**：新增 turnover_reports 表，管理申报→审核→生成分成账单流程。
13. **导入批次追踪**：新增 import_batches 表，支持整批回滚与部分导入两种策略。
14. **合同税费口径**：新增 tax*inclusive*、*applicable*tax_rate 字段，NOI 计算统一使用不含税口径。
15. **合同提前终止**：新增 termination_type 枚举及违约金、押金扣除明细等字段，WALE 中该合同剩余租期归零。
16. **信用评级量化**： tenants 表新增评级计算辅助字段（last*rating*date、times*overdue*past*12m*、*max*single*overdue*days），每月 1 日自动重算。
17. **KPI 反向指标**：kpi*metric*definitions 新增 direction 字段（positive/negative），反向指标线性插值逻辑翻转。
18. **单元参考市场租金**：units 新增 market*rent*reference、predecessor*unit*ids、archived_at 字段。

19. **刷新令牌**：新增 refresh_tokens 表，支持 JWT 续签、设备吊销、会话版本校验。
20. **递增规则模板**：新增 escalation_templates 表，支持递增规则保存为模板并从模板快速应用。
21. **楼层图纸版本管理**：新增 floor*plans* 表，支持同一楼层多版本图纸（改造前/后），`*is*current` 标识当前生效版本。
22. **PIPL 合规字段**：tenants 和 subleases 新增 `data_retention_until` ，合同终止后个人信息保留不超过 3 年。
23. **预警定向推送**：alerts 新增 `target_user_id` ，支持按用户级定向推送，不仅限于角色广播。
24. **子租赁审核草稿**：sublease*review*status 枚举新增 `'draft'` 状态，支持二房东填报暂存。
25. **合同初始状态**：contracts.status 默认值改为 `'quoting'` （报价中），与 PRD 合同生命周期一致。